

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

Đề tài: Data Platform for Marketing Analysis

Lớp: 168482
Học phần: Lưu trữ và xử lý dữ liệu lớn
Mã học phần: IT4931
Giảng viên hướng dẫn: **TS. Trần Việt Trung**

Danh sách thành viên nhóm 6:

Họ và tên	Mã số sinh viên
Phạm Đức Anh	20235008
Hoàng Đức Anh	20230015
Hồ Minh Dũng	20235050
Nguyễn Công Minh	20235161
Đặng Quốc Vinh	20235247

Hà Nội, tháng 6 năm 2026

MỤC LỤC

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN.....	4
1.1. Bài toán lựa chọn.....	4
1.2. Phân tích mức độ phù hợp với dữ liệu lớn.....	4
1.3. Phạm vi và giới hạn dự án.....	5
CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ.....	6
2.1. Kiến trúc tổng thể.....	6
2.2. Các thành phần và vai trò.....	7
2.2.1. Apache Airflow.....	7
2.2.2. Apache Kafka và Kafka Connect.....	7
2.2.3. MinIO.....	7
2.2.4. Apache Spark.....	7
2.2.5. ClickHouse.....	8
2.2.6. Apache Superset.....	8
2.3. Sơ đồ luồng dữ liệu và tương tác giữa các thành phần.....	8
CHƯƠNG 3. CHI TIẾT TRIỂN KHAI.....	8
3.1. Ingest dữ liệu Facebook, Tiktok, Google.....	8
3.2. Xử lý dữ liệu theo lô.....	9
3.2.1. Mục tiêu.....	9
3.2.2. Logic xử lý dữ liệu.....	9
3.2.3. Một số công thức tính chỉ số.....	10
3.3. Xử lý real time.....	11
3.3.1. Mục tiêu.....	11
3.3.2. Logic xử lý dữ liệu.....	11
3.4. Data Warehouse.....	12
3.4.1. Mục tiêu.....	12
3.4.2. Quy trình Data Modeling.....	12
3.4.3. Bus Architecture trong hệ thống.....	18
3.5. Điều phối Airflow.....	20
3.5.1. Các luồng DAG chính trong hệ thống.....	20
3.6. Cấu trúc tổ chức của DAG.....	20
3.6.1. Lập lịch chạy (Scheduling).....	21
3.7. Trực quan hóa trên Superset.....	22
3.7.1. Giới thiệu.....	22
3.7.2. Cấu trúc Dashboard và các chỉ số trực quan hóa.....	22
3.7.3. Kết quả.....	24
CHƯƠNG 4. Triển khai hệ thống.....	28
4.1. Docker.....	28

4.1.1. Các dịch vụ được triển khai.....	28
4.1.2. Chi tiết các dịch vụ.....	28
4.2. Minikube.....	29
4.2.1. Cấu trúc tài nguyên Kubernetes.....	29
4.2.2. Docker Images tùy biến.....	29
4.2.3. Chi tiết các dịch vụ.....	30
4.2.4. Quản lý tài nguyên.....	30
4.2.5. Truy cập giao diện.....	31
CHƯƠNG 5. BÀI HỌC KINH NGHIỆM.....	31
5.1. Thu thập dữ liệu.....	32
5.2. Xử lý dữ liệu với Spark.....	32
5.3. Lưu trữ dữ liệu.....	33
5.3.1. Mô tả vấn đề.....	33
5.3.2. Cách tiếp cận đã thử.....	33
5.3.3. Giải pháp cuối cùng.....	33
5.4. Giám sát & gỡ lỗi.....	33
5.4.1. Mô tả vấn đề.....	33
5.4.2. Cách tiếp cận đã thử.....	34
5.4.3. Giải pháp cuối cùng.....	35
5.4.4. Điểm rút ra.....	36
5.5. Mở rộng (Scaling).....	36
5.5.1. Mô tả vấn đề.....	36
5.5.2. Cách tiếp cận đã thử.....	37
5.5.3. Giải pháp cuối cùng.....	37
5.5.4. Điểm rút ra.....	38
5.6. Chất lượng dữ liệu & kiểm thử.....	39
5.7. Bảo mật & quản trị.....	39
5.7.1. Mô tả vấn đề.....	39
5.7.2. Cách tiếp cận đã thử.....	40
5.7.3. Giải pháp cuối cùng.....	40
5.7.4. Điểm rút ra.....	41
CHƯƠNG 6. NHẬN XÉT, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN.....	42
6.1. Nhận xét, đánh giá.....	42
6.2. Hướng phát triển.....	42
DANH MỤC TÀI LIỆU THAM KHẢO.....	43

CHƯƠNG 1. ĐỊNH NGHĨA BÀI TOÁN

1.1. Bài toán lựa chọn

Trong kỷ nguyên số hóa, mô hình bán hàng đa kênh (Omnichannel) ngày càng trở thành xu hướng tất yếu, giúp doanh nghiệp mở rộng khả năng tiếp cận khách hàng và tối ưu hiệu quả kinh doanh. Tuy nhiên, việc vận hành đồng thời nhiều nền tảng như Facebook Business Manager, TikTok Ads Manager, Google Ads và các hệ thống quản lý đơn hàng cũng đặt ra một thách thức lớn: dữ liệu bị phân tán, khó tổng hợp và khó khai thác kịp thời.

Thực tế này khiến đội ngũ vận hành quảng cáo phải tiêu tốn nhiều thời gian cho các công việc thủ công như đăng nhập nhiều hệ thống, tải dữ liệu, kiểm tra lỗi và tổng hợp báo cáo. Đồng thời, cấp quản lý cũng gặp khó khăn trong việc theo dõi hiệu quả chiến dịch theo thời gian thực, từ đó ảnh hưởng đến khả năng ra quyết định và phân bổ ngân sách quảng cáo. Đặc biệt trong các giai đoạn cao điểm như chiến dịch Sale lớn, việc chậm trễ hoặc sai lệch dữ liệu có thể gây ra nhiều tổn thất cho doanh nghiệp.

Từ những vấn đề trên, việc xây dựng một hệ thống tự động thu thập, lưu trữ và xử lý dữ liệu marketing từ nhiều nền tảng là cần thiết. Hệ thống không chỉ giúp giảm thiểu thao tác thủ công, hạn chế sai sót, mà còn hỗ trợ doanh nghiệp có cái nhìn tổng quan, chính xác và kịp thời hơn về hiệu quả hoạt động quảng cáo. Nhóm chúng em quyết định lựa chọn đề tài: Data Platform for Marketing Analysis.

1.2. Phân tích mức độ phù hợp với dữ liệu lớn

Bài toán tổng hợp và phân tích dữ liệu đa nền tảng này hoàn toàn phù hợp để áp dụng các công nghệ Dữ liệu lớn vì những lý do sau:

- **Dữ liệu phân tán và đa dạng:** Dữ liệu đến từ nhiều nền tảng (Facebook, Google, TikTok) với cấu trúc và định dạng khác nhau. Cần một hệ thống có khả năng thu thập, tích hợp và chuẩn hóa luồng dữ liệu đa dạng này về một cấu trúc chung để phân tích.
- **Tốc độ cập nhật (Velocity):** Dữ liệu quảng cáo yêu cầu tính cập nhật liên tục để phản ánh đúng hiện trạng chiến dịch. Hệ thống Big Data cung cấp khả năng xử lý dòng dữ liệu liên tục (Streaming) để có thể theo dõi hiệu quả theo thời gian thực (Real-time).
- **Khối lượng lớn và tăng trưởng nhanh (Volume):** Các chỉ số quảng cáo (hiển thị, nhấp chuột, chi phí) đi kèm với nhiều phân khúc chi tiết (độ tuổi, giới tính, địa điểm, từ khóa) sinh ra lượng dữ liệu rất lớn mỗi ngày. Các công nghệ lưu trữ

và xử lý phân tán sẽ đảm bảo hệ thống không bị quá tải và dễ dàng mở rộng khi cần.

1.3. Phạm vi và giới hạn dự án

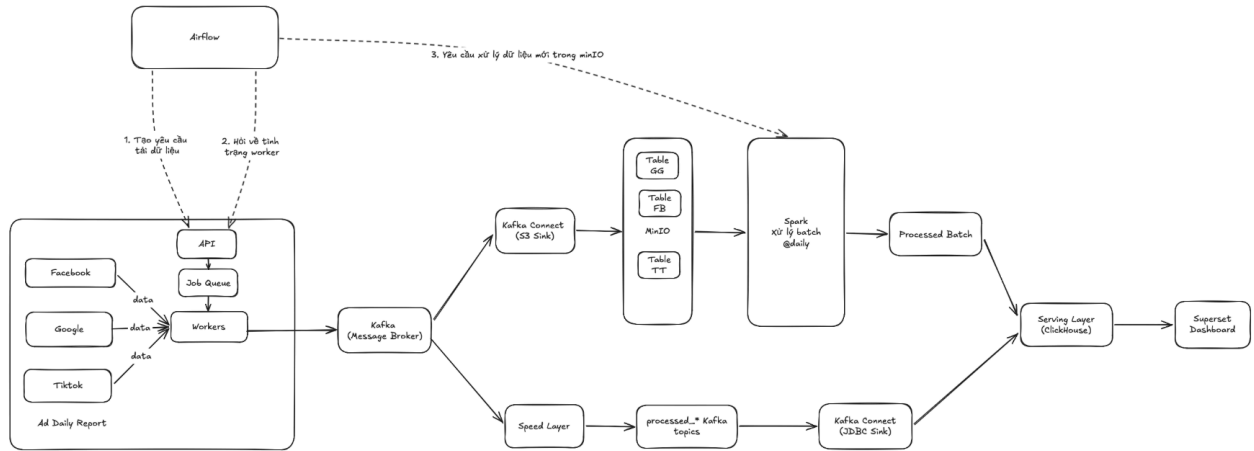
Trong phạm vi Bài tập lớn này, nhóm chúng em thực hiện xây dựng kiến trúc hệ thống thu thập và khai thác dữ liệu liên quan đến marketing trên các nền tảng Facebook, Google và TikTok.

- **Về mặt dữ liệu:** Để đảm bảo tính bảo mật và giữ bí mật thông tin của doanh nghiệp, thay vì gọi trực tiếp API từ các nền tảng thực tế, nhóm đã xây dựng module để khởi tạo bộ dữ liệu mock quy mô lớn. Dữ liệu mock được sinh ra sẽ có cơ cấu chi phí, phiếu chuyển đổi tham khảo từ dữ liệu thật và có sự biến động, tăng tiến liên tục theo thời gian để giả lập môi trường thực tế.
- **Về mặt triển khai:** Do nguồn lực có hạn, hệ thống được nhóm đóng gói và triển khai trên môi trường Docker/Minikube cục bộ, giả lập môi trường phân tán nhưng sẽ gặp một số hạn chế về mặt tài nguyên xử lý so với hệ thống Cluster thực tế.

Mặc dù nhóm đã cố gắng hoàn thiện sản phẩm, song không thể tránh khỏi những thiếu sót về kiến thức và quá trình kiểm thử. Nhóm chúng em rất mong nhận được những nhận xét, góp ý từ thầy để tiếp tục hoàn thiện đề tài tốt hơn. Cuối cùng, nhóm chúng em xin chân thành cảm ơn thầy TS. Trần Việt Trung đã hướng dẫn chúng em trong suốt quá trình thực hiện Bài tập lớn này.

CHƯƠNG 2. KIẾN TRÚC VÀ THIẾT KẾ

2.1. Kiến trúc tổng thể



Hình 2.1.1. Kiến trúc tổng thể hệ thống

Hệ thống được xây dựng gồm 4 phần với các chức năng nhằm thu thập, xử lý, lưu trữ và trực quan hoá dữ liệu marketing từ thông tin tuyển dụng trong trang web. Các thành phần của hệ thống bao gồm:

1. Bộ phận sinh dữ liệu: Dữ liệu được sinh giả lập với cấu trúc Ad Account → Campaign → Adset → Ad, mô phỏng dữ liệu trả về từ 3 nguồn Facebook Ads, TikTok Ads, Google Ads. Sau khi sinh, dữ liệu được đẩy lên Kafka; Kafka Connect tự động chuyển tiếp dữ liệu thô xuống MinIO (Data Lake).
2. Bộ phận lưu trữ: Hệ thống sử dụng MinIO làm Data Lake để lưu trữ toàn bộ dữ liệu thô dạng file JSON phân vùng theo ngày, cung cấp nguồn ổn định cho luồng xử lý theo lô. ClickHouse đóng vai trò Data Warehouse, lưu trữ dữ liệu sạch đã qua xử lý dưới dạng Star Schema (bảng Dimension và Fact) phục vụ truy vấn phân tích OLAP tốc độ cao.
3. Bộ phận xử lý dữ liệu: Spark đảm nhiệm hai luồng xử lý song song. Batch Layer đọc dữ liệu từ MinIO, làm sạch, chuẩn hoá và tách thành các bảng dim, fact rồi ghi vào ClickHouse theo lịch mỗi ngày một lần. Speed Layer sử dụng Spark Structured Streaming đọc trực tiếp từ Kafka, xử lý và ghi kết quả ra các Kafka processed topic; ClickHouse tự động tiêu thụ qua cơ chế Kafka Engine kết hợp Materialized View, cập nhật dữ liệu trong vòng vài chục giây.
4. Bộ phận biểu diễn dữ liệu: ClickHouse tổng hợp dữ liệu từ cả hai luồng — dữ liệu lịch sử chính xác từ Batch Layer và dữ liệu gần thời gian thực từ Speed Layer (lưu với TTL 1 ngày, sau đó được thay thế bởi kết quả Batch). Superset kết nối trực tiếp với

ClickHouse để trực quan hoá trên các Dashboard đa chiều, hỗ trợ cả phân tích tổng quan dài hạn lẫn giám sát hiệu suất tức thời.

2.2. Các thành phần và vai trò

2.2.1. Apache Airflow

Vai trò: Là bộ điều phối (Orchestrator) trung tâm của toàn bộ hệ thống.

Chức năng chính: Lập lịch, quản lý và giám sát tự động toàn bộ vòng đời của các luồng công việc (Data Pipeline), từ việc kích hoạt sinh dữ liệu giả lập, luân chuyển dữ liệu, cho đến việc điều phối các tác vụ tính toán phức tạp ở các tầng xử lý phía sau.

2.2.2. Apache Kafka và Kafka Connect

Vai trò: Truyền tải thông điệp (Message Broker) và trung chuyển dữ liệu.

Chức năng chính:

- Kafka: Tiếp nhận và phân phối dòng dữ liệu thô liên tục (streaming) từ các nguồn sinh dữ liệu, đảm bảo khả năng chịu tải cao và không bị mất mát dữ liệu.
- Kafka Connect: Đóng vai trò là cầu nối tự động, luân chuyển dữ liệu từ Kafka xuống Data Lake để lưu trữ thô, hoặc đẩy trực tiếp dữ liệu đã qua xử lý vào Data Warehouse mà không cần can thiệp thủ công.

2.2.3. MinIO

Vai trò: Là hệ thống lưu trữ dữ liệu dạng đối tượng (Object Storage), đóng vai trò là Data Lake của hệ thống.

Chức năng chính: Cung cấp "Landing Zone" (vùng đáp) dung lượng lớn để lưu trữ dài hạn toàn bộ dữ liệu thô. Việc này giúp tách biệt tầng thu thập và tầng xử lý, đồng thời cung cấp nguồn dữ liệu ổn định cho luồng xử lý theo lô (Batch) và lưu trữ điểm khôi phục (checkpoint) cho luồng thời gian thực.

2.2.4. Apache Spark

Vai trò: Là engine tính toán phân tán trung tâm thực hiện các tác vụ ETL/ELT.

Chức năng chính: Đảm nhiệm việc đọc dữ liệu từ các vùng lưu trữ (Kafka hoặc MinIO), sau đó làm sạch, chuẩn hóa, bóc tách và biến đổi dữ liệu thô thành cấu trúc có ý nghĩa (các bảng Dimension và Fact) cho cả luồng xử lý theo lô và theo thời gian thực.

2.2.5. ClickHouse

Vai trò: Là hệ quản trị cơ sở dữ liệu phân tích trực tuyến (OLAP Database), đóng vai trò là Data Warehouse.

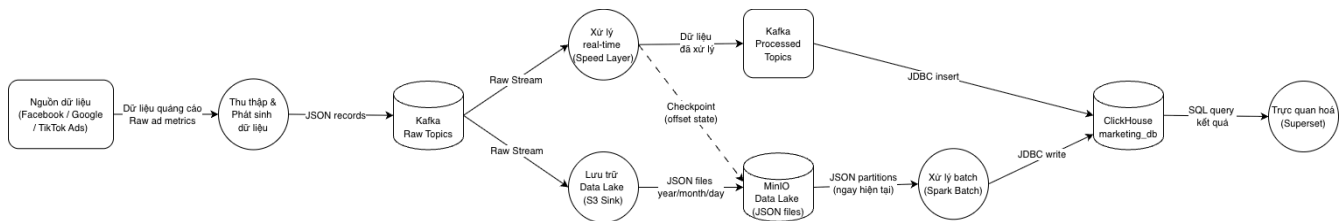
Chức năng chính: Nhờ kiến trúc lưu trữ dạng cột (Columnar Storage), ClickHouse lưu trữ toàn bộ dữ liệu sạch cuối cùng của hệ thống. Nó cung cấp tốc độ truy vấn tổng hợp cực kỳ nhanh, đáp ứng tối ưu cho các tác vụ phân tích, gom nhóm (Group By) dữ liệu Marketing theo nhiều chiều khác nhau.

2.2.6. Apache Superset

Vai trò: Là lớp giao diện phân tích kinh doanh (Presentation Layer) tương tác trực tiếp với người dùng cuối.

Chức năng chính: Kết nối trực tiếp với ClickHouse để trực quan hóa dữ liệu thành các dạng biểu đồ, bảng biểu. Superset cung cấp các Dashboard đa chiều giúp nhà quản lý dễ dàng theo dõi hiệu suất chiến dịch theo cả hai góc nhìn: tổng quan dài hạn và giám sát tức thời (Real-time).

2.3. Sơ đồ luồng dữ liệu và tương tác giữa các thành phần



Hình 2.3.1. Sơ đồ luồng dữ liệu

CHƯƠNG 3. CHI TIẾT TRIỂN KHAI

3.1. Ingest dữ liệu Facebook, Tiktok, Google

Để giữ bí mật dữ liệu của doanh nghiệp, thay vì gọi API thực tế, chúng em đã xây dựng module để khởi tạo bộ dữ liệu mock. Dữ liệu mock sẽ có cơ cấu chi phí và phiếu chuyển đổi tham khảo từ dữ liệu thực tế.

Dữ liệu mock sẽ có nhiều level từ Ad account, Campaign, Adset, Ad và breakdown theo ngày, theo tuổi, theo giới tính và địa điểm. Dữ liệu Google còn có breakdown theo keyword để track được hiệu suất của từ khóa tìm kiếm.

Dữ liệu được sinh trong ngày đảm bảo không đổi với các trường id, tên đối tượng và tăng tiến với các chỉ số theo thời gian, đúng với tính cập nhật liên tục của nền tảng thực tế.

3.2. Xử lý dữ liệu theo lô

3.2.1. Mục tiêu

Mục tiêu của luồng xử lý dữ liệu theo lô (Batch Layer) là đọc dữ liệu từ các file trong MinIO, sau đó thực hiện biến đổi, chuẩn hoá, tách thành các bảng Dimension và bảng Fact để lưu vào trong Warehouse (ở đây là ClickHouse). Batch Layer nhằm ghi lại dữ liệu chính xác cuối cùng, sửa lại những lỗi có thể sinh ra trong Speed Layer.

Batch Layer được chạy theo từng ngày, mỗi lần chạy chỉ scan lại dữ liệu trong ngày chứ không thực hiện scan lại toàn bộ dữ liệu.

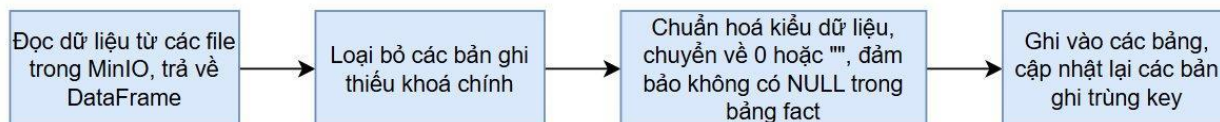
3.2.2. Logic xử lý dữ liệu

Khi Batch Layer được thực thi, các bảng trong MinIO sẽ được đọc, xử lý và ghi vào các bảng trong ClickHouse. Cụ thể các bảng tương ứng như sau:

Bảng nguồn MinIO	Bảng target ClickHouse
fad_ad_daily_report	fad_ad_daily_report, dim_account, dim_campaign, dim_adset, dim_ad, dim_creative, dim_date, fact_fb_ad_daily, fact_fb_ad_creative_daily
fad_age_gender_detailed_report	fact_fb_ad_demographic_daily
gad_campaign_daily_report	gad_campaign_daily_report, fact_gg_campaign_daily
gad_ad_group_daily_report	gad_ad_group_daily_report, fact_gg_adgroup_daily
gad_account_daily_report	gad_account_daily_report, dim_account
gad_keyword_performance_report	gad_keyword_performance_report, dim_campaign, dim_gg_adgroup, fact_gg_keyword_daily
gad_age_report	gad_age_report, fact_gg_age_daily
gad_gender_report	gad_gender_report, fact_gg_gender_daily
gad_ad_asset_daily_report	gad_ad_asset_daily_report, dim_gg_asset, fact_gg_asset_daily

gad_click_type_report	gad_click_type_report, fact_gg_click_type_daily
tta_ad_performance	tta_ad_performance, dim_tta_advertiser, dim_tta_ad, fact_tta_ad_daily

Đối với từng bảng trên, Batch Layer cùng thực hiện theo một pattern như sau:



3.2.3. Một số công thức tính chỉ số

Để phục vụ việc theo dõi một cách hiệu quả dữ liệu quảng cáo từ các nền tảng, một số chỉ số đặc thù được tính khi xử lý để đưa vào ClickHouse như sau:

Chỉ số	Công thức	Ý nghĩa
CTR (Click-Through Rate)	Clicks / Impressions	Tỷ lệ người nhấp vào quảng cáo trên tổng lượt hiển thị
CPC (Cost Per Click)	Spend / Clicks	Chi phí trung bình cho mỗi lần nhấp
CPM (Cost Per Mille)	$(\text{Spend} / \text{Impressions}) \times 1000$	Chi phí cho 1.000 lần hiển thị
Frequency	Impressions / Reach	Số lần trung bình một người thấy quảng cáo
Conversion Rate	Conversions / Clicks	Tỷ lệ chuyển đổi trên số lần nhấp
Cost per Conversion	Spend / Conversions	Chi phí để đạt được 1 conversion
ROAS (Return on Ad Spend)	Revenue / Spend	Doanh thu thu về trên mỗi đồng chi quảng cáo

3.3. Xử lý real time

3.3.1. Mục tiêu

Mục tiêu của luồng xử lý dữ liệu thời gian thực (Speed Layer) là tiêu thụ dữ liệu ngay khi xuất hiện trên Kafka, thực hiện biến đổi, chuẩn hoá và tách thành các bảng Dimension và bảng Fact để cập nhật vào ClickHouse trong vòng vài chục giây. Speed Layer bù đắp khoảng trống về thời gian của Batch Layer: trong khi Batch Layer chạy theo lịch mỗi ngày một lần, Speed Layer đảm bảo dashboard phản ánh dữ liệu gần như tức thì, phục vụ nhu cầu theo dõi hiệu suất quảng cáo trong ngày.

Dữ liệu do Speed Layer tạo ra được lưu với TTL (thời gian sống) là 1 ngày - sau đó tự động được thay thế bởi dữ liệu chính xác hơn từ Batch Layer. Speed Layer được trigger theo chu kỳ 30 giây - mỗi chu kỳ gom toàn bộ bản ghi mới xuất hiện trong khoảng thời gian đó, xử lý và ghi kết quả ra Kafka. ClickHouse tự động tiêu thụ kết quả thông qua cơ chế Kafka Engine kết hợp Materialized View, không cần thao tác trung gian.

2.1.1. Logic xử lý dữ liệu

Khi Speed Layer được thực thi, dữ liệu từ các topic Kafka thô sẽ được đọc, xử lý và kết quả được ghi vào các bảng real-time trong ClickHouse (ký hiệu tiền tố rt_). Cụ thể các bảng tương ứng như sau:

Topic nguồn (Kafka)	Bảng target ClickHouse
fad_ad_daily_report	rt_fad_ad_daily_report, rt_dim_account, rt_dim_campaign, rt_dim_adset, rt_dim_ad, rt_dim_creative, rt_dim_date, rt_fact_fb_ad_daily, rt_fact_fb_ad_creative_daily
fad_age_gender_detailed_report	rt_fact_fb_ad_demographic_daily
gad_campaign_daily_report	rt_gad_campaign_daily_report, rt_dim_gg_campaign, rt_fact_gg_campaign_daily
gad_ad_group_daily_report	rt_gad_ad_group_daily_report, rt_dim_gg_adgroup, rt_fact_gg_adgroup_daily
gad_account_daily_report	rt_gad_account_daily_report
gad_keyword_performance_report	rt_gad_keyword_performance_report, rt_fact_gg_keyword_daily

gad_age_report	rt_gad_age_report, rt_fact_gg_age_daily
gad_gender_report	rt_gad_gender_report, rt_fact_gg_gender_daily
gad_ad_asset_daily_report	rt_gad_ad_asset_daily_report, rt_dim_gg_asset, rt_fact_gg_asset_daily
gad_click_type_report	rt_gad_click_type_report, rt_fact_gg_click_type_daily
tta_ad_performance	rt_tta_ad_performance, rt_dim_tta_advertiser, rt_dim_tta_ad, rt_fact_tta_ad_daily

Pattern xử lý cho từng topic tương tự Batch Layer, có 2 điểm khác biệt chính:

- Nguồn đọc là Kafka Streaming thay vì file trên MinIO
- Đích ghi là các Kafka processed topic thay vì ghi trực tiếp vào ClickHouse

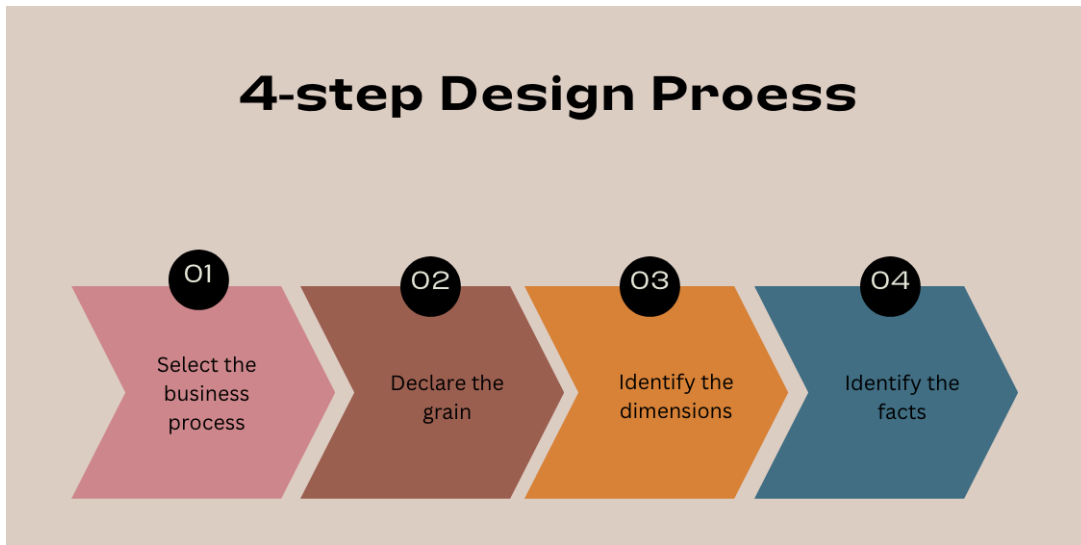
3.4. Data Warehouse

3.4.1. Mục tiêu

Data Warehouse trong hệ thống này phục vụ serving layer cho cả Batch Layer và Speed Layer trong kiến trúc hệ thống. Trong dự án này, ClickHouse được lựa chọn để làm Data Warehouse. Lý do cho sự lựa chọn này là do hầu hết các truy vấn từ Superset đều là truy vấn OLAP (Online Analytical Processing) (chẳng hạn như SUM(spend), AVG(ctr), GROUP BY account/date...). ClickHouse là một columnar database, giúp các truy vấn này được thực hiện nhanh chóng hơn các row-oriented database như MySQL, PostgreSQL...

3.4.2. Quy trình Data Modeling

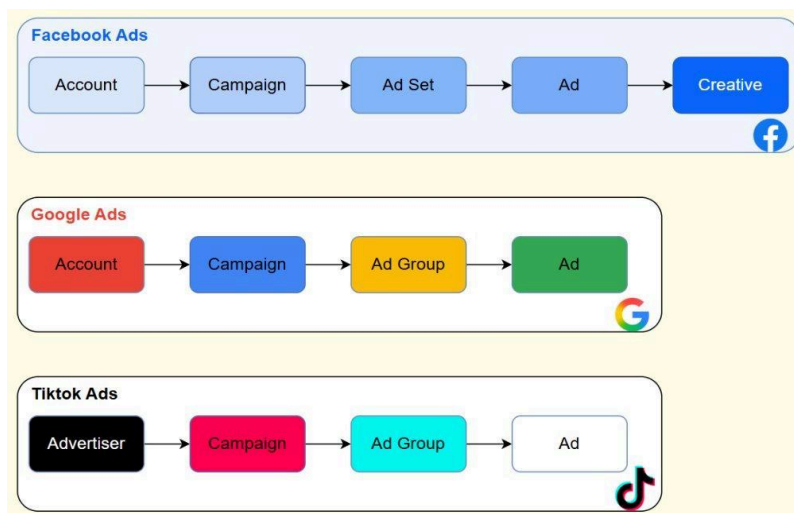
Để xác định cấu trúc các bảng (schema) trong Warehouse, nhóm đã thực hiện quy trình Data Modeling. Quy trình Data Modeling được tiến hành theo 4 bước như sau:



a. Xác định quy trình nghiệp vụ (Select the Business Process)

Quy trình nghiệp vụ: Quản lý hiệu suất các chiến dịch quảng cáo trên các nền tảng online.

Hệ thống quản lý dữ liệu quảng cáo từ 3 nền tảng Facebook, Google, Tiktok, mỗi nền tảng có cấu trúc phân cấp riêng như sau:



Mỗi cấp độ trong phân cấp dữ liệu trên sẽ ứng với 1 fact table. Nhiều fact table chia sẻ chung một số bảng *conformed dimensions*.

b. Xác định độ chi tiết (Declare the Grain)

Ở đây, mỗi Grain sẽ định nghĩa chính xác mỗi dòng trong bảng fact đại diện cho điều gì.

Fact Table	Grain	ORDER BY trong Clickhouse
------------	-------	---------------------------

fact_fb_ad_daily	1 quảng cáo × 1 ngày	(account_id, ad_id, date_start)
fact_fb_ad_creative_daily	1 creative × 1 quảng cáo × 1 ngày	(account_id, ad_id, creative_id, date_start)
fact_fb_ad_demographic_daily	1 (age × gender) × 1 quảng cáo × 1 ngày	(account_id, ad_id, age, gender, date_start)
fact_gg_campaign_daily	1 campaign × 1 ngày	(campaign_id, date)
fact_gg_adgroup_daily	1 nhóm quảng cáo × 1 ngày	(adgroup_id, date)
fact_gg_keyword_daily	1 keyword × 1 device × 1 ngày	(account_id, adgroup_id, keyword, device, date)
fact_gg_age_daily	1 age range × 1 device × 1 ngày	(account_id, adgroup_id, age_range, device, date)
fact_gg_gender_daily	1 gender × 1 device × 1 ngày	(account_id, adgroup_id, gender, device, date)
fact_gg_asset_daily	1 asset × 1 quảng cáo × 1 ngày	(account_id, adgroup_id, ad_id, asset_id, date)
fact_gg_click_type_daily	1 click type × 1 device × 1 ngày	(account_id, campaign_id, click_type, device, date)
fact_tta_ad_daily	1 quảng cáo × 1 ngày	(advertiser_id, ad_id, date)

c. Xác định các bảng dimension (Identify the Dimensions)

Các bảng dimension được xác định như sau:

Dimension Table	Mô tả	Sử dụng cho?
dim_date	Lịch: năm, quý, tháng, tuần, ngày, tên ngày, cuối tuần, ngày lễ VN	Tất cả
dim_account	Tài khoản quảng cáo (account_id, account_name)	Facebook + Google
dim_campaign	Chiến dịch (campaign_id, account_id, campaign_name)	Facebook + Google
dim_adset	Ad Set Facebook (adset_id, campaign_id, adset_name)	Facebook

dim_ad	Ad Facebook (ad_id, adset_id, ad_name, status, created_time)	Facebook
dim_creative	Creative Facebook (creative_id, title, thumbnail_url, link, page)	Facebook
dim_gg_adgroup	Ad Group Google (adgroup_id, campaign_id, adgroup_name)	Google
dim_gg_asset	Asset Google (asset_id, ad_id, type, text, image_url)	Google
dim_tta_advertiser	Nhà quảng cáo TikTok (advertiser_id, advertiser_name)	Tiktok
dim_tta_ad	Ad TikTok (ad_id, advertiser_id, campaign_name, adgroup_name, ad_name, ad_text)	Tiktok

Ở đây, dim_date và dim_account là *conformed dimensions* được nhiều fact table từ nhiều platform khác nhau cùng tham chiếu. Đây là cơ chế cho phép drill-across giữa các fact table: ví dụ so sánh Facebook và Google cùng account_id, hoặc phân tích tất cả platform theo cùng dim_date.

d. Xác định các bảng fact (Identify the Facts)

Mỗi fact table tương ứng với 1 grain cùng các khoá ngoại (FK) trỏ tới các bảng dimension và các chỉ số đo lường (measures). Schema các bảng được xác định chi tiết như sau:

Facebook Ads

fact_fb_ad_daily - hiệu suất quảng cáo theo ngày

Loại cột	Cột	Mô tả
FK	date_start	Ngày
FK	account_id	Tài khoản
FK	ad_id	Quảng cáo
Measure	spend	Tổng chi phí
Measure	impressions, reach	Hiển thị/người dùng
Measure	clicks, link_clicks, landing_page_views	Funnel nhấp chuột

Measure	ctr, cpc, cpm, frequency	Hiệu suất
Measure	new_messaging_connections, cost_per_new_messaging	Chỉ số nhắn tin

fact_fb_ad_creative_daily - hiệu suất creative gắn với quảng cáo theo ngày

Loại cột	Cột	Mô tả
FK	date_start, account_id, ad_id	Ngày / tài khoản / quảng cáo
FK	creative_id	Creative
Measure	spend, impressions, reach, clicks	Delivery cơ bản
Measure	new_messaging_connections	Nhắn tin
Measure	post_engagements, post_reactions, post_shares, photo_views	Tương tác bài viết

fact_fb_ad_demographic_daily - phân tách hiệu suất theo nhân khẩu học

Loại cột	Cột	Mô tả
FK	date_start, account_id, ad_id	Ngày / tài khoản / quảng cáo
Degenerate dim	age, gender	Nhân khẩu học
Measure	spend, impressions, reach, clicks	Delivery cơ bản
Measure	inline_link_clicks, new_messaging_connections	Tương tác

Google Ads

fact_gg_campaign_daily - hiệu suất cấp campaign theo ngày

Loại cột	Cột	Mô tả
FK	date, adgroup_id	Ngày / group quảng cáo
Measure	impressions, clicks, cost, ctr, all_conversions	Delivery + tỉ lệ chuyển đổi

fact_gg_keyword_daily - hiệu suất từ khóa, phân tách theo device

Loại cột	Cột	Mô tả
FK	date, account_id, campaign_id, adgroup_id	Ngày + phân cấp
Degenerate dim	keyword, device	Từ khóa và thiết bị
Measure	impressions, clicks, cost, ctr	Delivery
Measure	conversions, all_conversions, cost_per_conversion	Chuyển đổi
Measure	average_cpc, quality_score	Chất lượng keyword

fact_gg_age_daily / fact_gg_gender_daily - phân tách theo nhân khẩu học

Loại cột	Cột	Mô tả
FK	date, account_id, campaign_id, adgroup_id	Ngày + phân cấp
Degenerate dim	age_range / gender, device	Nhân khẩu học và thiết bị
Measure	impressions, clicks, cost, ctr, conversions, all_conversions, average_cpc, cost_per_conversion	Delivery + chuyển đổi

fact_gg_asset_daily - hiệu suất creative asset gắn với quảng cáo

Loại cột	Cột	Mô tả
FK	date, account_id, campaign_id, adgroup_id, ad_id, asset_id	Ngày + phân cấp
Degenerate dim	asset_performance	Xếp hạng Google (BEST / GOOD / LOW)
Measure	impressions, clicks, cost, ctr, all_conversions	Delivery + chuyển đổi

fact_gg_click_type_daily - phân tách click theo loại và network

Loại cột	Cột	Mô tả
FK	date, account_id, campaign_id	Ngày + campaign
Degenerate dim	click_type, device, ad_network_type	Loại click + thiết bị + mạng quảng cáo
Measure	impressions, clicks, cost, ctr, conversions, all_conversions	Delivery + chuyển đổi

Tiktok Ads

fact_tta_ad_daily - hiệu suất quảng cáo theo ngày

Loại cột	Cột	Mô tả
FK	date, advertiser_id, ad_id	Ngày + nhà quảng cáo + quảng cáo
Measure	spend, impressions, reach, clicks	Delivery
Measure	ctr, cpc, cpm, frequency	Hiệu suất tổng hợp
Measure	conversion, cost_per_conversion, conversion_rate	Chuyển đổi
Measure	video_play_actions, profile_visits, likes, comments, shares, follows, live_views	Engagement TikTok
Measure	purchase, onsite_shopping, total_onsite_shopping_value, onsite_shopping_roas, cost_per_onsite_shopping	Shopping

Ở đây, *degenerate dimension* là các cột phân loại (như age, gender, keyword, device, click_type) nằm trực tiếp trong fact table thay vì được tách ra bằng bảng dim riêng. Chúng không có thuộc tính bổ sung nào để lưu trữ nên không cần bảng riêng.

3.4.3. Bus Architecture trong hệ thống

Mỗi hàng là một quy trình nghiệp vụ kèm fact table tương ứng. Mỗi cột là một dimension table.

Quy trình nghiệp vụ	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)	(10)
Facebook										
Hiệu suất quảng cáo theo ngày	x	x	x	x	x					
Hiệu suất creative theo ngày	x	x	x	x	x	x				
Phân tích nhân khẩu học	x	x	x	x	x					
Google										
Hiệu suất campaign theo ngày	x	x	x							
Hiệu suất nhóm quảng cáo theo ngày	x	x	x							
Hiệu suất từ khoá	x	x	x				x			
Phân tích độ tuổi	x	x					x			
Phân tích giới tính	x	x					x			
Hiệu suất asset quảng cáo	x	x	x				x	x		
Phân tích click	x	x	x							
Tiktok										
Hiệu suất	x								x	x

quảng cáo										
-----------	--	--	--	--	--	--	--	--	--	--

Chú thích các bảng:

- (1): dim_date
- (2): dim_account
- (3): dim_campaign
- (4): dim_adset
- (5): dim_ad
- (6): dim_creative
- (7): dim_gg_adgroup
- (8): dim_gg_adset
- (9): dim_tta_advertiser
- (10): dim_tta_ad

3.5. Điều phối Airflow

3.5.1. Các luồng DAG chính trong hệ thống

Hệ thống hiện tại sử dụng 3 luồng DAG chính trong Airflow:

a. marketing_data_pipeline:

- Là luồng tích hợp batch end-to-end.
- Quy trình: Mock data → Kafka → (Kafka Connect ghi MinIO) → Spark ingest → Clickhouse.

b. marketing_mock_data_generation:

- Chỉ thực hiện sinh dữ liệu mock (Facebook/Google/TikTok) và đẩy vào Kafka.
- Luồng này không chạy bước Spark ingest vào ClickHouse.

c. marketing_spark_ingestion:

- Chỉ chạy Spark batch ingest từ MinIO sang ClickHouse.
- Được sử dụng khi cần nạp dữ liệu độc lập với bước tạo mock.

3.6. Cấu trúc tổ chức của DAG

Mỗi DAG được tổ chức theo kiến trúc 3 lớp rõ ràng:

- Khai báo DAG: Bao gồm định nghĩa các thuộc tính cơ bản như start_date, schedule_interval, catchup, tags, và max_active_runs.
- Khai báo Task: Chủ yếu sử dụng BashOperator để chạy các lệnh Python/Spark bên trong container.
- Khai báo Dependency (Thứ tự chạy): Thiết lập thứ tự thực thi của các task bằng cách sử dụng toán tử “>>” để nối luồng.

Cấu trúc task (dependency) thực tế của 3 luồng DAG được cập nhật như sau:

a. marketing_data_pipeline

Luồng: setup_dependencies → (mock_generation_facebook, mock_generation_google, mock_generation_tiktok (song song)) → wait_kafka_connect_flush → minio_to_clickhouse_ingest

b. marketing_mock_data_generation

Luồng: setup_dependencies → (mock_generation_facebook, mock_generation_google, mock_generation_tiktok (song song)) → wait_kafka_connect_flush.

c. marketing_spark_ingestion

Luồng: minio_to_clickhouse_ingest.

3.6.1. Lập lịch chạy (Scheduling)

Việc lập lịch cụ thể cho các DAG được thiết lập như sau:

DAG ID	Tần suất lập lịch (schedule_interval)	catchup	Mục đích
marketing_data_pipeline	@daily	False	Chạy tích hợp batch end-to-end 1 lần/ngày.
marketing_mock_data_generation	*/15 * * * *	False	Chạy mỗi 15 phút để tạo dữ liệu mock gần realtime.
marketing_spark_ingestion	@daily	False	Chạy batch ingest độc lập mỗi ngày.

Nguyên tắc Scheduling đang áp dụng:

- catchup=False: Được áp dụng để tránh Airflow tự động tạo hàng loạt run lịch sử, giúp ngăn ngừa tình trạng quá tải tài nguyên.
- Backfill dữ liệu lịch sử: Khi cần nạp lại dữ liệu quá khứ, việc kích hoạt được thực hiện có kiểm soát theo từng ngày (tuần tự) để tránh tràn RAM và ngăn chặn việc nạp cộng dồn dữ liệu.

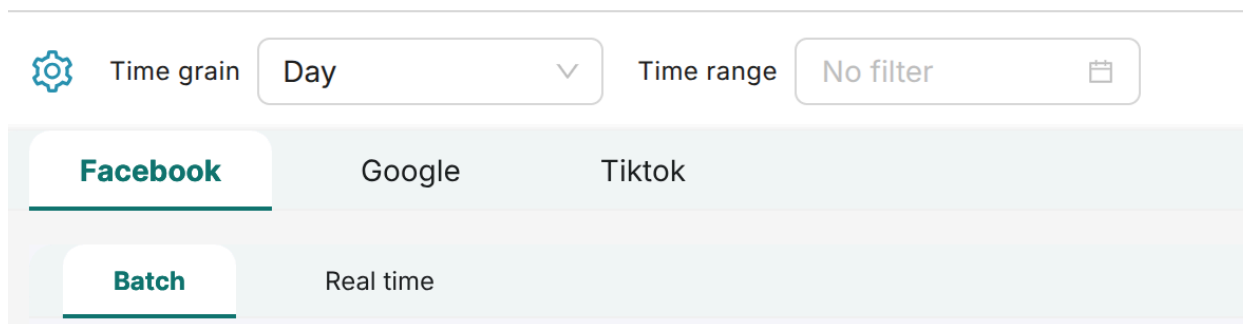
3.7. Trục quan hóa trên Superset

3.7.1. Giới thiệu

Superset được sử dụng làm công cụ Trục quan hóa (BI Tool) trong hệ thống, kết nối trực tiếp với Data Warehouse ClickHouse. Chức năng chính là hỗ trợ các truy vấn OLAP (Online Analytical Processing) như tính tổng chi phí (SUM(spend)), trung bình tỉ lệ nhấp chuột (AVG(ctr)) và nhóm theo tài khoản/ngày (GROUP BY account/date) để phân tích hiệu suất quảng cáo một cách nhanh chóng.

3.7.2. Cấu trúc Dashboard và các chỉ số trục quan hóa

Dashboard trên Superset được thiết kế với cấu trúc 3 lớp nền tảng chính (Facebook, Google, TikTok) và 2 tab con và các bộ lọc tương tác



a. Bộ lọc tương tác (Interactive Filters)

Để tăng tính linh hoạt và khả năng phân tích dữ liệu trên Dashboard, Superset còn được trang bị hai bộ lọc thời gian chính:

- **Time Grain:** Cho phép người dùng tùy chỉnh độ chi tiết của dữ liệu hiển thị, ví dụ: lọc theo từng ngày, từng tuần, hoặc từng tháng. Việc thay đổi Time Grain giúp chuyển đổi nhanh chóng giữa phân tích hiệu suất chi tiết và đánh giá xu hướng dài hạn.
- **Time Range:** Cho phép người dùng chọn một khoảng thời gian cụ thể để xem dữ liệu (ví dụ: 7 ngày qua, tháng trước, hoặc một phạm vi ngày tùy chỉnh), hỗ trợ việc phân tích so sánh và xác định hiệu suất trong các giai đoạn cụ thể.

b. Nội dung các Tab Batch

Các tab Batch truy vấn từ các bảng Fact và Dimension trong ClickHouse, cung cấp cái nhìn đa chiều và chi tiết về hiệu suất quảng cáo.

Chỉ số đo lường chính (Measures):

- **Delivery:** Spend (Tổng chi phí), Impressions (Lượt hiển thị), Reach (Người dùng tiếp cận).
- **Phễu chuyển đổi:** Clicks (Số lần nhấp), Conversions (Số lượt chuyển đổi), Landing Page Views (Lượt xem trang đích).

Chỉ số hiệu suất tổng hợp: Các chỉ số được tính toán trong quá trình xử lý dữ liệu theo lô để dễ dàng theo dõi:

- CTR (Click-Through Rate)
- CPC (Cost Per Click)
- CPM (Cost Per Mille)
- Frequency (Tần suất)
- ROAS (Return on Ad Spend)

Phân tích Chiều sâu (Dimensions): Cho phép phân tích hiệu suất theo các chiều (dimensions) được định nghĩa trong Data Modeling:

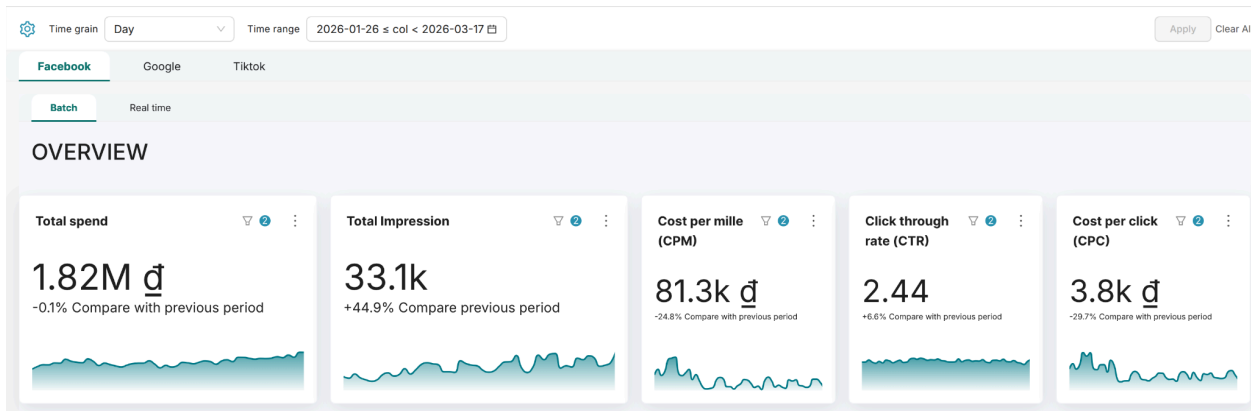
- Thời gian: Phân tích theo ngày, tuần, tháng, quý (sử dụng dim_date).
- Phân cấp quảng cáo: Tài khoản (dim_account), Chiến dịch (dim_campaign), Nhóm quảng cáo (Adset/Adgroup), và Quảng cáo (Ad).
- Nhân khẩu học: Phân tích theo Độ tuổi (Age) và Giới tính (Gender) (ví dụ: fact_fb_ad_demographic_daily, fact_gg_age_daily).
- Creative: Phân tích hiệu suất Creative trên Facebook (dim_creative) và Asset trên Google (dim_gg_asset).

c. Nội dung các Tab Realtime

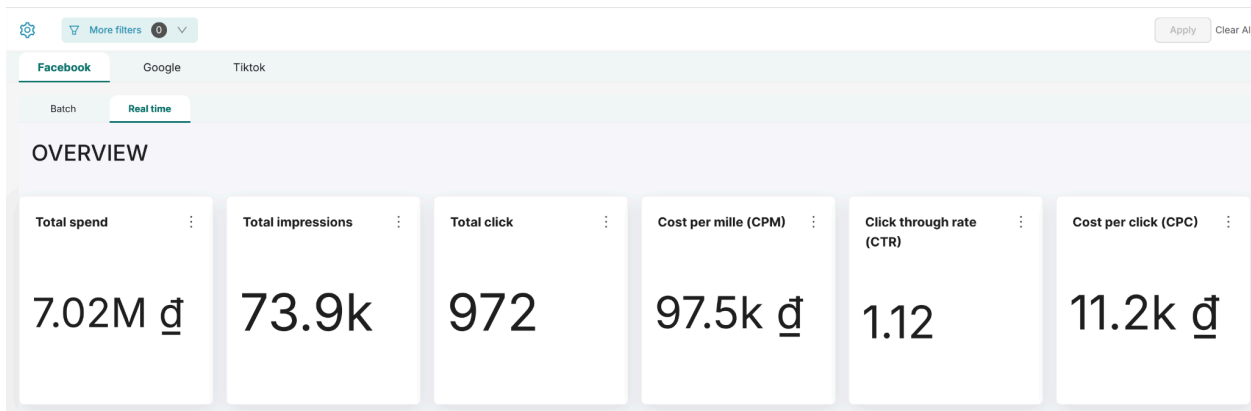
Các tab Realtime tập trung vào việc giám sát nhanh chóng các chỉ số Delivery và Costing quan trọng, đặc biệt cần thiết để kiểm soát các chiến dịch Sale lớn nhằm tránh "đốt" ngân sách và bỏ lỡ "thời điểm vàng". Các chỉ số thường được cập nhật theo chu kỳ ngắn (ví dụ: 15 phút) để phản ánh tình hình hoạt động gần nhất của quảng cáo.

3.7.3. Kết quả

Các biểu đồ Big Number đóng vai trò là "Quick Overview" và được đặt ở vị trí nổi bật tại phần trên cùng của mỗi Dashboard (cả tab Batch và Realtime). Chức năng của chúng là giúp người dùng nắm bắt sơ lược về tình hình hoạt động cốt lõi của từng nền tảng (Facebook, Google, TikTok) thông qua các chỉ số đo lường chính (KPIs) trong phạm vi thời gian được chọn.

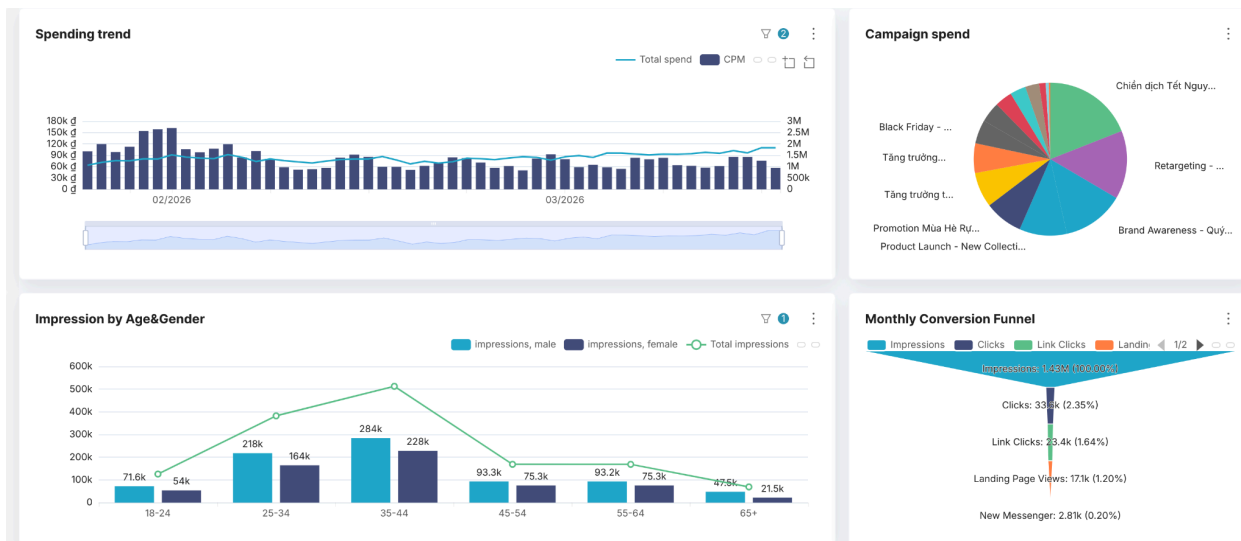


Ảnh 1: Overview Batch Dashboard

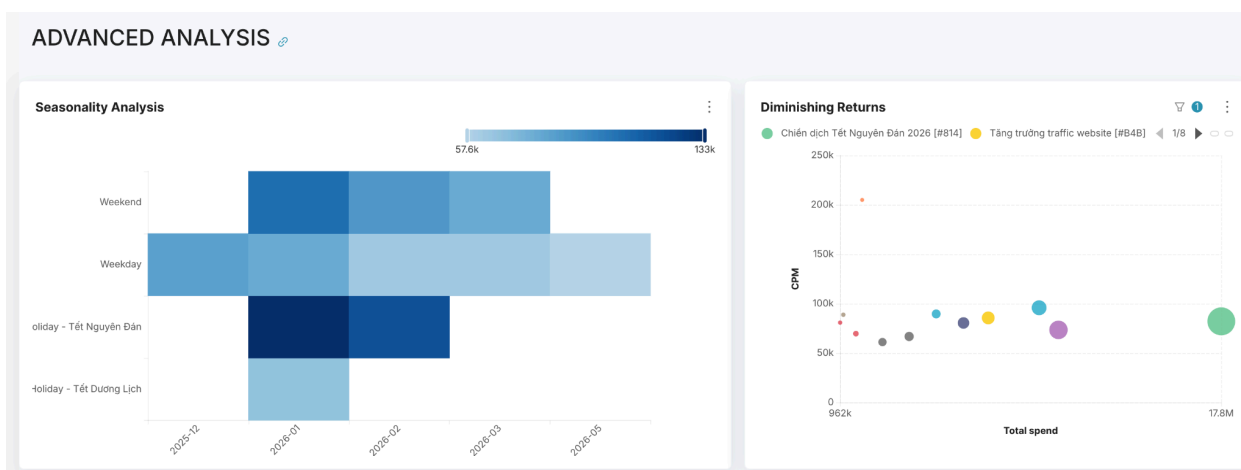


Ảnh 2: Overview Real-time Dashboard

Ngoài ra, dashboard còn sử dụng các biểu đồ trực quan nâng cao như Heatmap để làm rõ các chỉ số cao hơn vào cuối tuần và ngày lễ, và Bubble Chart để biểu diễn hiện tượng Lợi suất giảm dần (Diminishing Returns) đối với Total Spend và CPM.



Ảnh 3: Các charts trong Batch Dashboard Facebook

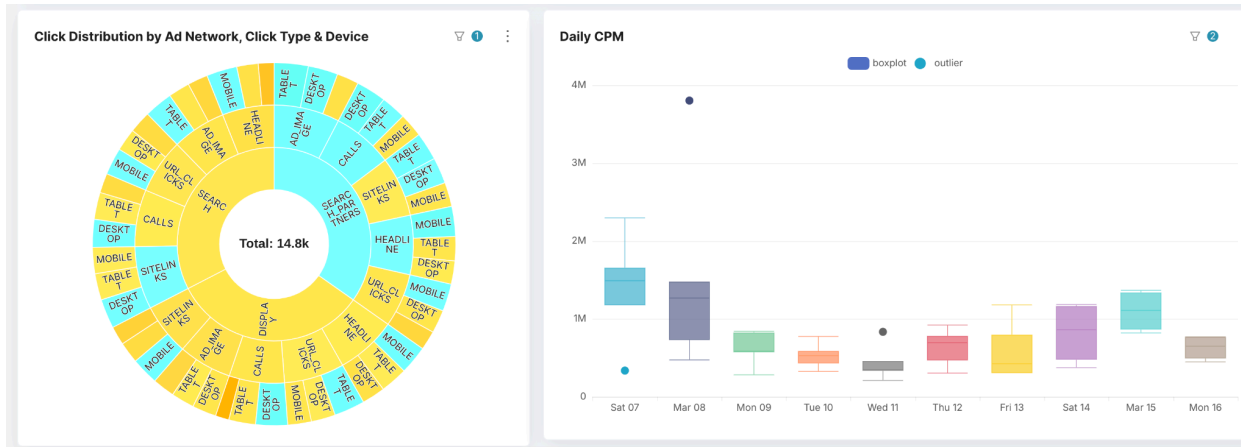


Ảnh 4: Các charts trong Batch Dashboard Facebook

Đặc biệt, đối với dữ liệu của Google, dashboard sử dụng thêm Word Cloud để trực quan hóa các xu hướng từ khóa tìm kiếm (keyword) đang được sử dụng và có hiệu suất cao, giúp người dùng nắm bắt nhanh chóng các yếu tố tìm kiếm nổi bật. Ngoài ra, các công cụ trực quan nâng cao được áp dụng bao gồm Sunburst Chart để biểu diễn phân bố của các Click chuột theo cấu trúc phân cấp, Box Plot để thể hiện phân bố và các điểm ngoại lai của CPM hàng ngày.

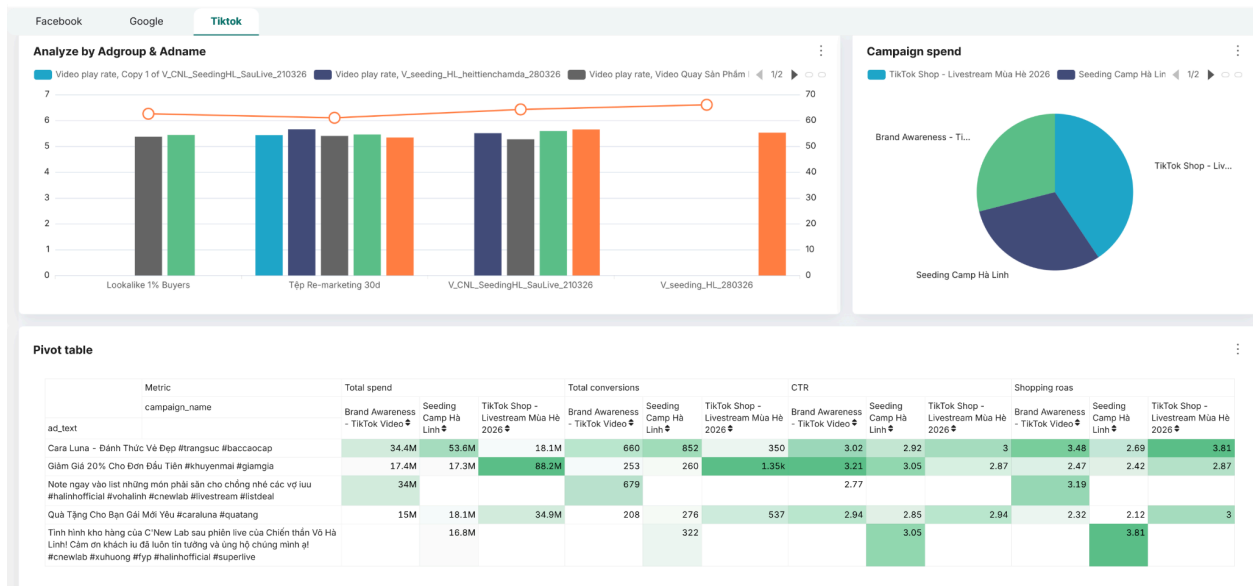


Ảnh 5: Các charts trong Batch Dashboard Google



Ảnh 6: Các charts trong Batch Dashboard Google

Với nền tảng TikTok, dashboard sử dụng Mix Chart (Bar & Line) để biểu diễn Engagement Rate, tức là tỉ lệ người dùng tương tác với video tiktok (mỗi lượt like, share, comment được tính là tương tác), và Pivot Table để xem hiệu quả đầu tư chi tiết.



Ảnh 7: Các charts trong Batch Dashboard Tiktok

CHƯƠNG 4. Triển khai hệ thống

4.1. Docker

4.1.1. Các dịch vụ được triển khai

Để dễ dàng trong việc kiểm thử và phát triển, hệ thống được đóng gói thành một mạng docker-compose, gồm các dịch vụ sau: MinIO, ClickHouse, PostgreSQL, Kafka, Kafka Connect, Batch Consumer, Spark, Airflow, Superset, KafkaUI.

4.1.2. Chi tiết các dịch vụ

Hệ thống sử dụng Apache Kafka chạy ở chế độ KRaft với 1 node, giúp đơn giản hóa kiến trúc và tăng hiệu năng:

- kafka: Chạy trên cổng 9092. Đóng vai trò làm xương sống (backbone) cho luồng dữ liệu thời gian thực.
- kafka-ui: Giao diện Web (Cổng 8084) giúp quản trị viên dễ dàng theo dõi broker, topic, connector.
- kafka-connect-init: Script khởi tạo kết nối thông qua REST API của Kafka Connect.
- kafka-connect: Đảm nhiệm việc tải dữ liệu từ Kafka xuống MinIO và Clickhouse. Hệ thống tự động cài đặt và đăng ký 2 connector:
 - o S3 Sink Connector: Đẩy dữ liệu từ Kafka vào MinIO.
 - o JDBC Sink Connector: Đẩy dữ liệu vào ClickHouse.

Để lưu trữ dữ liệu thô, hệ thống dùng MinIO trên cổng 9005 và 9006. Dữ liệu trong MinIO được lưu vào bucket marketing_datalake và partition theo ngày để Spark đọc và xác định dữ liệu nào đã xử lý, dữ liệu nào chưa.

Clickhouse tại cổng 8123 và 9000 đảm nhiệm lưu trữ dữ liệu sạch, có sẵn script khởi tạo star schema để dữ liệu sạch đổ vào và sẵn sàng cho analytics.

Airflow tại cổng 8082 đảm nhiệm việc điều phối, gồm 4 service PostgreSQL (Airflow metadata database), airflow-init (khởi tạo cấu hình airflow), airflow-webserver (web UI), airflow-scheduler (kích hoạt các DAG để chạy theo cấu hình thời gian đã định sẵn).

Service speed-layer sử dụng Spark Structured Streaming, trigger mỗi 30s để lấy dữ liệu từ topic kafka. Dữ liệu sẽ được xử lý và đưa vào clickhouse. Speed-layer khi kích hoạt sẽ submit Spark job với yêu cầu tài nguyên 4 core CPU và 512mb RAM.

Để đơn giản và tiết kiệm tài nguyên khi chạy trên 1 máy, Spark cluster trong mạng docker compose chỉ gồm 1 service master và 1 service worker.

4.2. Minikube

Ngoài môi trường Docker Compose phục vụ phát triển và kiểm thử, hệ thống còn được triển khai lên Minikube — một cụm Kubernetes chạy cục bộ trên máy tính cá nhân. So với Docker Compose, Minikube bổ sung khả năng quản lý tài nguyên (giới hạn RAM/CPU cho từng dịch vụ), tự động khởi động lại khi dịch vụ gặp sự cố và mô phỏng sát hơn môi trường triển khai thực tế trên cloud.

4.2.1. Cấu trúc tài nguyên Kubernetes

Toàn bộ tài nguyên được tổ chức trong một namespace duy nhất tên marketing. Hệ thống sử dụng đầy đủ các loại tài nguyên Kubernetes:

- ConfigMap: lưu các biến cấu hình không nhạy cảm của Kafka, ClickHouse, Airflow, Superset.
- Secret: lưu thông tin xác thực (mật khẩu, Fernet key) của MinIO, ClickHouse, PostgreSQL, Airflow.
- PersistentVolumeClaim (PVC): cấp phát bộ nhớ lưu trữ bền vững cho 5 dịch vụ có trạng thái: Kafka, ClickHouse, MinIO, PostgreSQL, Superset. Dữ liệu trong PVC được giữ nguyên sau khi pod khởi động lại.
- Deployment: định nghĩa 12 dịch vụ với số lượng bản sao và cấu hình tài nguyên.
- Service: cung cấp địa chỉ mạng cố định để các pod liên lạc với nhau, và mở cổng NodePort để truy cập giao diện web từ máy host.
- Job: 3 tác vụ khởi tạo chạy một lần duy nhất khi hệ thống được triển khai lần đầu.
- RBAC: phân quyền đặc biệt cho Airflow Scheduler được phép tắt/bật Speed Layer trong quá trình chạy pipeline.

4.2.2. Docker Images tùy biến

Khác với Docker Compose có thể mount code trực tiếp từ máy host, Kubernetes yêu cầu toàn bộ code và cấu hình phải được đóng gói sẵn bên trong image. Dự án xây dựng 3 image tùy biến cho mục đích này:

- mkt_airflow: tích hợp Airflow, Spark, toàn bộ mã nguồn Python và các thư viện JAR cần thiết cho Spark Streaming (Kafka connector, S3/MinIO connector). Image này được

tái sử dụng cho cả Airflow Scheduler, Airflow Webserver, Speed Layer và Batch Consumer.

- mkt_superset: Superset được cài sẵn driver kết nối ClickHouse.
- mkt_clickhouse: ClickHouse được nhúng sẵn các script SQL khởi tạo Star Schema (batch tables) và Real-time tables, tự động chạy khi container khởi động.

4.2.3. Chi tiết các dịch vụ

Các dịch vụ triển khai trên Minikube tương đương với Docker Compose về chức năng, nhưng có một số điểm khác biệt trong cấu hình:

- Kafka chạy ở chế độ KRaft (không dùng ZooKeeper), dữ liệu topic được lưu bền vững vào PVC 5 GB. Kafka được cấu hình hai listener riêng biệt: một cho giao tiếp nội bộ giữa các pod trong cluster (kafka:29092), một cho truy cập từ bên ngoài qua NodePort.
- Kafka Connect cần bộ nhớ lớn hơn các dịch vụ khác (giới hạn 3 GB) do phải tải đồng thời S3 Sink plugin, JDBC plugin và ClickHouse JDBC driver khi khởi động. Sau khi sẵn sàng, một Job khởi tạo tự động đăng ký hai connector (S3 Sink và JDBC Sink) qua REST API.
- Speed Layer chạy như một Deployment độc lập, thực thi lệnh spark-submit kết nối tới Spark Master trong cluster. Điểm đáng chú ý là Spark Driver cần quảng bá địa chỉ IP thật của pod (spark.driver.host) để Spark Executor có thể kết nối ngược lại — khác với môi trường Docker Compose nơi hostname được giải quyết tự động.
- Airflow được tách thành hai Deployment riêng (Scheduler và Webserver). Airflow Scheduler được cấp quyền RBAC đặc biệt để có thể gọi Kubernetes API scale Speed Layer về 0 bản sao trước khi chạy batch ingest, sau đó khôi phục lại — tránh tranh chấp tài nguyên Spark Worker giữa hai luồng xử lý.

4.2.4. Quản lý tài nguyên

Mỗi Deployment được cấu hình hai mức tài nguyên. Request là lượng tài nguyên Kubernetes đặt chỗ khi xếp lịch pod - tổng request của tất cả dịch vụ ở mức 2,1 CPU và 5,25 GB RAM, chiếm khoảng 40% tài nguyên của máy ảo Minikube (4 CPU, 8 GB). Limit là ngưỡng tối đa một pod được phép sử dụng - vượt quá giới hạn RAM sẽ bị hệ thống dừng và

khởi động lại, vượt giới hạn CPU sẽ bị giảm tốc. Việc để tổng limit lớn hơn capacity thực tế là chủ ý, các dịch vụ không bao giờ đạt mức tối đa cùng lúc nên không gây xung đột.

4.2.5. Truy cập giao diện

Các giao diện web được mở ra ngoài thông qua NodePort. Địa chỉ truy cập có dạng `http://<minikube-ip>:<nodeport>`, trong đó <minikube-ip> lấy bằng lệnh `minikube ip`.

Dịch vụ	NodePort	Tài khoản mặc định
Airflow	30082	admin / password123
Superset	30088	admin / password123
MinIO Console	30006	admin / password123
Spark Master UI	30081	
ClickHouse HTTP	30123	admin / password123

CHƯƠNG 5. BÀI HỌC KINH NGHIỆM

5.1. Thu thập dữ liệu

Nhóm chọn dữ liệu Marketing, với đa dạng 3 nền tảng. Mỗi nền tảng có cấu trúc JSON trả về khác nhau. Nhưng phía doanh nghiệp chỉ cung cấp dữ liệu đã che đi các thông tin nhạy cảm như tên Ads, tên chiến dịch và thông tin của các quảng cáo, không có Access Token để gọi lên nền tảng.

Nhóm có thể sử dụng luôn dữ liệu phía doanh nghiệp để đưa vào hệ thống, dữ liệu này đảm bảo phản ánh đúng thông tin marketing thực tế nhưng khối lượng nhỏ, không đủ để test thông lượng của pipeline.

Nhóm đã chọn hướng dựa trên những insight phân tích từ dữ liệu doanh nghiệp để tạo bộ sinh dữ liệu riêng. Có thể dữ liệu này chưa phản ánh đủ quy luật thị trường nhưng nhóm có thể tùy ý điều chỉnh khối lượng dữ liệu để test thông lượng pipeline.

5.2. Xử lý dữ liệu với Spark

a. Dùng bộ nhớ tạm thì phải trả lại

Khi Spark xử lý một tập dữ liệu qua nhiều bước khác nhau, nó cho phép giữ dữ liệu trung gian trong bộ nhớ để không phải tính lại từ đầu; kỹ thuật này tiết kiệm thời gian đáng kể. Tuy nhiên nếu sau khi dùng xong không "trả lại" bộ nhớ đó, Spark sẽ tích lũy dần và dẫn đến tình trạng thiếu bộ nhớ khi xử lý các batch tiếp theo. Do vậy, cần phải giải phóng bộ nhớ khi bộ nhớ tích lũy quá nhiều. Nguyên tắc đơn giản: mượn xong phải trả.

b. Streaming cần điểm khôi phục để không mất dữ liệu

Với luồng streaming, Spark đọc dữ liệu từ Kafka và ghi nhớ đã đọc đến đâu thông qua một "điểm đánh dấu" lưu trên MinIO. Khi hệ thống gặp sự cố và khởi động lại, Spark đọc điểm đánh dấu đó và tiếp tục từ chỗ dừng để không bị đọc lại từ đầu, không mất dữ liệu. Nếu xóa điểm đánh dấu này đi, Spark sẽ đọc lại toàn bộ lịch sử và nhồi trùng dữ liệu vào hệ thống, khiến ClickHouse phải mất thời gian xử lý dữ liệu trùng.

c. Image nặng là chi phí ẩn

Vì tích hợp cả Spark vào image Airflow để tiện deploy, image trở nên rất nặng — mỗi lần build và chuyển vào Kubernetes tiêu tốn hàng gigabyte dung lượng ổ đĩa. Đây là lý do máy liên tục hết chỗ trong quá trình thử nghiệm. Về lâu dài, tách Spark ra thành một dịch vụ riêng biệt sẽ giúp image Airflow nhẹ hơn nhiều và dễ cập nhật độc lập.

5.3. Lưu trữ dữ liệu

5.3.1. Mô tả vấn đề

Cần xác định một định dạng lưu trữ trong minIO để phía spark có thể xử lý, cần có cơ chế để spark biết được dữ liệu nào đã xử lý, dữ liệu nào chưa.

5.3.2. Cách tiếp cận đã thử

Dữ liệu lưu trữ trên minIO bằng Parquet, cơ chế hoạt động của Kafka Connect sẽ flush các message sau một khoảng thời gian hoặc khi đủ buffer. Nếu cần lưu 10000 message vào minIO, với Kafka Connect set 1000 limit cho buffer thì sẽ tạo ra 10 file Parquet độc lập trên minIO. Cách này giúp spark xử lý tốt hơn nhưng tốn tài nguyên khi ghi ở Kafka connect do cần build cấu trúc cho từng file một. Lý do nhóm chọn limit là 1000 vì không phải topic nào cũng nhiều dữ liệu, nếu set quá ít thì bị nhiều file, set quá nhiều thì có topic rất lâu mới flush xuống minIO gây tốn RAM không cần thiết.

Nhóm cũng đã thử lưu bằng JSON với cấu hình Kafka Connect tương tự. JSON có ưu điểm đơn giản và dễ debug hơn, đỡ tốn CPU khi flush. Đổi lại tốc độ xử lý của spark giảm.

Nhóm có hai hướng cấu trúc đường dẫn partition theo ngày và theo giờ.

5.3.3. Giải pháp cuối cùng

Nhóm đã chạy thử nghiệm và nhận thấy việc lưu bằng JSON giảm thời gian chạy của DAG end - to - end hơn so với dùng Parquet.

Vì luồng spark xử lý batch chỉ cần chạy theo ngày (mỗi ngày một lần) nên chỉ cần cấu hình partition theo ngày là đủ.

5.4. Giám sát & gỡ lỗi

5.4.1. Mô tả vấn đề

Bối cảnh và nền tảng

AdsCrawler là hệ thống khai thác dữ liệu marketing tích hợp nhiều thành phần hạ tầng như Airflow, Kafka, Kafka Connect, MinIO, Spark, ClickHouse và Superset. Do vận hành đồng thời cả hai luồng xử lý theo lô (batch) và thời gian thực (realtime), công tác giám sát yêu cầu khả năng theo dõi liên tục toàn bộ hành trình dữ liệu từ khâu khởi tạo, vận chuyển đến lưu trữ cuối cùng.

Thách thức gặp phải

Hiện tại, việc quản trị hệ thống vẫn dựa trên giao diện rời rạc của Airflow, Spark, Kafka UI và truy xuất log qua *kubectl logs*. Quy trình vận hành còn tồn tại các thao tác thủ công thiếu

tối ưu, điển hình như việc sử dụng lệnh trì hoãn *sleep 90* để chờ Kafka Connect hoàn tất đẩy dữ liệu.

Dựa trên log ghi nhận, hệ thống phát sinh một số lỗi vận hành tiêu biểu như:

- Task Airflow thất bại do tiến trình *pip install* bị gián đoạn kết nối mạng.
- Spark không thể truy xuất dữ liệu khi tệp tin trên MinIO chưa được khởi tạo kịp thời.
- Cạn kiệt tài nguyên lưu trữ (*No space left on device*) khi thực hiện nạp dữ liệu lịch sử quy mô lớn.
- Sự cố mất kết nối giữa Airflow và Postgres dẫn đến việc task bị dừng đột ngột bởi tín hiệu *SIGTERM*.

Tác động đến hệ thống

Những hạn chế trên gây ra các rủi ro hệ thống đáng kể, khiến pipeline dễ dàng bị gián đoạn dù logic nghiệp vụ vẫn đảm bảo chính xác:

- Tần suất thất bại và thử lại (retry) của DAG tăng cao.
- Dữ liệu batch bị thiếu hụt do tiến trình nạp chạy trước khi dữ liệu kịp ghi xuống.
- Tốn thời gian truy vết nguyên nhân do phải phân tích log thủ công.
- Thiếu cơ chế cảnh báo sớm đối với các lỗi hệ thống nghiêm trọng.

5.4.2. Cách tiếp cận đã thử

Theo dõi qua UI và log thủ công

Nhóm tận dụng các công cụ quản trị sẵn có như Airflow, Spark, Kafka UI và nhật ký vận hành từ container. Đây là phương thức chủ đạo đang được vận hành.

Đánh đổi:

- Ưu điểm: Khai thác trực tiếp hạ tầng hiện hữu mà không cần thiết lập thêm.
- Nhược điểm: Thông tin bị phân mảnh, gây khó khăn cho việc đối chiếu và thiếu cơ chế chủ động thông báo sự cố.

Sử dụng cơ chế tự phục hồi sẵn có

Hệ thống thiết lập cấu hình *retries=1* trong Airflow, chế độ tự khởi động *on-failure* của Docker và chính sách *OnFailure* cho các Job trên Kubernetes.

Đánh đổi:

- Ưu điểm: Giúp giảm thiểu tác động của các lỗi tạm thời thông qua việc tự động hóa phục hồi.

- Nhược điểm: Chỉ giải quyết triệu chứng bề nổi, không thể xác định căn nguyên cốt lõi và không ngăn chặn được lỗi tái diễn.

Ghi nhật ký chi tiết trong mã nguồn

Các tiến trình xử lý được bổ sung log chi tiết về lưu lượng dữ liệu (dòng đọc/ghi), trạng thái bỏ qua bảng hoặc lỗi đẩy tin nhắn lên Kafka.

Đánh đổi:

- Ưu điểm: Hỗ trợ đắc lực cho việc rà soát và kiểm chứng từng công đoạn xử lý dữ liệu.
- Nhược điểm: Nhật ký thiếu cấu trúc chuẩn hóa, chưa tích hợp correlation ID, gây khó khăn cho việc tổng hợp thành các chỉ số vận hành có ý nghĩa.

5.4.3. Giải pháp cuối cùng

Giải pháp chi tiết

Hướng tiếp cận tối ưu nhất cho hệ thống là thiết lập mô hình giám sát tập trung 3 lớp:

- Lớp chỉ số (Metrics): Theo dõi hiệu suất tài nguyên (CPU, RAM, đĩa cứng), độ trễ Kafka, lưu lượng DLQ, thời lượng thực thi DAG và năng suất xử lý của từng job.
- Lớp cảnh báo: Kích hoạt thông báo khi phát hiện lỗi task liên tục, ngưỡng tài nguyên tới hạn, độ trễ Kafka tăng đột biến hoặc sự cố thiếu hụt dữ liệu trên MinIO.
- Lớp nhật ký và RCA: Chuẩn hóa định dạng log với đầy đủ ngữ cảnh vận hành (*dag_id, task_id, run_id, topic, process_date*), chuyển đổi cơ chế chờ sang kiểm tra trạng thái thực tế.

Triển khai đề xuất

Các cải tiến ưu tiên triển khai dựa trên hiện trạng mã nguồn bao gồm:

- Tích hợp sẵn mọi phụ thuộc vào image, loại bỏ thao tác *pip install* trong quá trình thực thi task.
- Thay thế lệnh trì hoãn bằng quy trình tự động xác thực sự tồn tại của tệp tin/phân vùng trên MinIO.
- Thiết lập giám sát chuyên biệt cho dung lượng lưu trữ và các thư mục tạm thời.
- Bổ sung cơ chế quản lý heartbeat giữa Airflow và Postgres để ngăn chặn lỗi dừng task bất ngờ.

Chỉ số và kết quả kỳ vọng

Hệ thống sau khi nâng cấp dự kiến sẽ đạt được các mục tiêu sau:

- Triệt tiêu các lỗi do phụ thuộc vào kết nối mạng ngoại vi trong lúc vận hành.
- Tối ưu hóa độ tin cậy của luồng nạp dữ liệu theo lô.
- Nhận diện sớm các rủi ro về hạ tầng và kết nối nội mạng.
- Đẩy nhanh tốc độ xử lý và khắc phục sự cố hệ thống.

5.4.4. Điểm rút ra

Hiểu biết kỹ thuật

Việc phân tích cho thấy lỗi trong các hệ thống phân tán thường có nguồn gốc từ các yếu tố hạ tầng (DNS, lưu trữ, mạng nội bộ) và sự thiếu đồng nhất trạng thái giữa các dịch vụ, thay vì chỉ nằm ở logic xử lý.

Thực hành tốt

Các nguyên tắc quản trị cần tuân thủ:

- Cấm cài đặt package trong thời gian chạy thực tế.
- Sử dụng kiểm tra trạng thái thực tế thay cho các khoảng chờ cố định.
- Phân định rạch ròi giữa chỉ số, cảnh báo và nhật ký vận hành.
- Giám sát song song cả hạ tầng và chất lượng dữ liệu đầu ra.

Đề xuất

Nhóm tập trung vào ba trọng tâm:

- Tập trung hóa việc theo dõi và chuẩn hóa log.
- Thiết lập hệ thống cảnh báo chủ động cho tài nguyên và tiến trình lỗi.
- Chuyển đổi sang các bước kiểm tra sức khỏe hệ thống (health check) thực tế.

5.5. Mở rộng (Scaling)

5.5.1. Mô tả vấn đề

a. Bối cảnh và nền tảng

AdsCrawler vận hành đồng thời 12 dịch vụ trên một cụm Minikube duy nhất với tài nguyên giới hạn (4 CPU, 12 GB RAM). Trong quá trình triển khai, nhóm nhận thấy hệ thống thường xuyên rơi vào trạng thái không ổn định: các pod bị OOMKill đột ngột, Spark Worker không còn tài nguyên để nhận job mới, và máy ảo Minikube liên tục báo hết dung lượng đĩa.

b. Thách thức gặp phải

Thách thức cốt lõi là hai luồng xử lý — Speed Layer và Batch Layer — tranh nhau tài nguyên của Spark Worker duy nhất trong cluster. Speed Layer chạy liên tục 24/7 để xử lý stream, trong khi Batch Layer cần một lượng lớn CPU và RAM trong khoảng thời gian ngắn để

thực thi spark-submit. Khi cả hai cùng hoạt động, Spark Worker bị quá tải, job batch thất bại hoặc chạy cực kỳ chậm.

Bên cạnh đó, Kafka Connect cần tải đồng thời ba plugin lớn (S3 Sink, JDBC Sink, ClickHouse JDBC driver) khi khởi động, tiêu thụ gần 3 GB RAM chỉ trong giai đoạn init — vượt xa mức tiêu thụ bình thường khi đã ổn định. Nếu không giới hạn đúng cách, pod này có thể ăn hết RAM của toàn node, kéo các pod khác vào trạng thái Pending.

c. Tác động đến hệ thống

Thiếu quy hoạch tài nguyên dẫn đến các hệ quả trực tiếp: DAG batch thất bại do Spark không đủ core, pod bị OOMKill giữa chừng khiến dữ liệu ghi dở, và máy ảo liên tục hết đĩa do image nặng tích lũy qua nhiều lần build thử nghiệm.

5.5.2. Cách tiếp cận đã thử

a. Không giới hạn tài nguyên (Docker Compose style)

Giai đoạn đầu nhóm chạy Minikube mà không khai báo requests/limits, để các pod tự do sử dụng tài nguyên như Docker Compose.

Đánh đổi:

- Ưu điểm: cấu hình đơn giản, không cần ước tính trước nhu cầu của từng service.
 - Nhược điểm: một pod bị leak memory hoặc chạy nặng có thể ăn hết tài nguyên node, kéo toàn bộ hệ thống sập mà không có cảnh báo. Kubernetes cũng không thể xếp lịch pod hợp lý nếu không biết pod cần bao nhiêu tài nguyên.
- b. Thêm requests/limits nhưng chưa xử lý xung đột Spark

Nhóm bổ sung khai báo resources cho từng Deployment, nhưng chưa có cơ chế phối hợp giữa Speed Layer và Batch Layer. Cả hai vẫn cùng tranh Spark Worker.

Đánh đổi:

- Ưu điểm: ngăn được tình trạng một pod ăn hết tài nguyên, K8s có thể xếp lịch ổn định hơn.
- Nhược điểm: xung đột Spark vẫn xảy ra — batch job chạy chậm bất thường hoặc bị timeout do Speed Layer đang giữ core.

5.5.3. Giải pháp cuối cùng

a. Chiến lược overcommit có kiểm soát

Nhóm áp dụng mô hình hai mức tài nguyên: requests đặt ở mức thấp (tổng 40% capacity) để Kubernetes có thể schedule toàn bộ 12 pod lên cùng một node, trong khi limits đặt cao hơn nhiều để các pod như Kafka Connect (cần 3 GB lúc khởi động) hay Airflow Scheduler (cần 2

GB khi submit Spark job) có đủ bộ nhớ thực tế. Overcommit an toàn vì các pod không bao giờ đạt đỉnh cùng lúc.

b. Cơ chế scale-to-zero để tránh tranh chấp Spark

Giải pháp triệt để cho xung đột Spark là tích hợp logic điều phối vào chính DAG: trước khi Batch Layer chạy, Airflow Scheduler gọi Kubernetes API để scale Speed Layer về 0 bản sao, giải phóng toàn bộ core cho batch job. Sau khi batch hoàn tất — kể cả khi thất bại — Speed Layer được khôi phục về 1 bản sao. Cơ chế này đòi hỏi Airflow Scheduler được cấp quyền RBAC tối thiểu: chỉ được phép get, patch, update trên deployments và deployments/scale trong namespace marketing, không có quyền nào khác.

c. Phân bổ PVC đúng nhu cầu

Dung lượng lưu trữ được cấp phát theo vai trò thực tế của từng service: MinIO nhận 10 GB (Data Lake lưu toàn bộ dữ liệu thô), Kafka và ClickHouse mỗi service 5 GB, PostgreSQL 2 GB cho metadata Airflow, Superset 1 GB cho cấu hình dashboard. Tổng 23 GB phù hợp với phạm vi đồ án.

5.5.4. Điểm rút ra

a. Hiểu biết kỹ thuật

Trong môi trường nhiều service chạy chung một node, tài nguyên là tài nguyên dùng chung — không quy hoạch từ đầu thì tranh chấp là tất yếu. Đặc biệt với Spark, mô hình driver-executor yêu cầu một lượng RAM và CPU đột biến trong thời gian ngắn, rất khác với các service chạy ổn định như web server hay database.

b. Thực hành tốt

Một số nguyên tắc nhóm rút ra được:

- Khai báo requests/limits ngay từ đầu, không để đến khi xảy ra sự cố mới điều chỉnh.
- Requests phản ánh mức tiêu thụ bình thường, limits phản ánh mức đỉnh — hai con số này thường chênh nhau đáng kể với các service có tải biến động như Spark.
- Khi hai luồng xử lý dùng chung tài nguyên, cần có cơ chế phối hợp rõ ràng thay vì để chúng tự cạnh tranh.

c. Đề xuất

Nếu triển khai lên môi trường production thực tế, ba cải tiến quan trọng nhất về scaling là: tách Spark thành cluster độc lập (không dùng chung với Airflow), bổ sung HorizontalPodAutoscaler dựa trên Kafka consumer lag, và thay MinIO bằng managed object storage (GCS, S3) để loại bỏ chi phí quản trị storage server.

5.6. Chất lượng dữ liệu & kiểm thử

Để củng cố tính toàn vẹn của dữ liệu và hiệu suất luồng công việc, nhóm đã thiết lập các kịch bản kiểm thử sau:

- Kiểm soát logic khởi tạo dữ liệu: Xác thực tính đồng nhất của định danh (ID) dựa trên seed sinh dữ liệu; đảm bảo sự biến động dữ liệu phản ánh đúng xu hướng thị trường và các điểm bùng nổ trong dịp lễ; kiểm chứng sự tương quan giữa từ khóa và nhân khẩu học (ví dụ: tác động của từ khóa giới tính lên phân bổ chi phí quảng cáo).
- Kiểm tra tính ổn định của DAG: Loại bỏ các rủi ro về lỗi nhập thư viện và xác nhận sự hiện diện của DAG trong hệ thống điều phối.

Dù các biện pháp trên đã giúp đẩy nhanh tiến độ gỡ lỗi, nhóm nhận thấy cần phát triển thêm các kịch bản kiểm thử chuyên sâu cho quy trình biến đổi dữ liệu (transformation) và các điều kiện thực tế khắt khe hơn để đạt tới tiêu chuẩn dữ liệu sản xuất chuyên nghiệp.

5.7. Bảo mật & quản trị

5.7.1. Mô tả vấn đề

a. Bối cảnh và nền tảng

Hệ thống AdsCrawler bao gồm nhiều dịch vụ có thông tin xác thực riêng biệt: Airflow, Superset, MinIO, ClickHouse, PostgreSQL, Kafka Connect đều yêu cầu username/password để hoạt động. Trong giai đoạn phát triển, nhóm ưu tiên tốc độ triển khai hơn bảo mật — toàn bộ thông tin xác thực được hardcode trực tiếp vào file cấu hình, không có mã hóa kết nối, và không có cơ chế kiểm soát truy cập giữa các service.

b. Thách thức gặp phải

Thách thức chính không phải là bị tấn công từ bên ngoài (hệ thống chỉ chạy local), mà là quản lý độ phức tạp nội bộ: khi một service cần thông tin xác thực của service khác, thông tin đó nằm ở đâu, ai được phép đọc, và khi đổi mật khẩu thì cần sửa bao nhiêu chỗ. Ngoài ra, Airflow Scheduler cần quyền gọi Kubernetes API để scale Speed Layer — nếu cấp quyền quá rộng, một lỗi trong DAG có thể vô tình xóa hoặc sửa các tài nguyên không liên quan.

c. Tác động đến hệ thống

Trong giai đoạn phát triển, cách tiếp cận đơn giản này không gây ra vấn đề nghiêm trọng. Tuy nhiên nhóm nhận thấy rõ hai rủi ro tiềm ẩn: thông tin xác thực xuất hiện ở nhiều nơi khác nhau trong codebase khiến việc thay đổi dễ bỏ sót, và không có nhật ký tập trung để biết luồng dữ liệu đi qua các service như thế nào khi cần truy vết sự cố.

5.7.2. Cách tiếp cận đã thử

a. Hardcode credentials trong file cấu hình

Giai đoạn đầu toàn bộ thông tin xác thực được viết trực tiếp vào docker-compose.yml và các file YAML của Kubernetes dưới dạng plaintext.

Đánh đổi:

- Ưu điểm: triển khai nhanh, dễ đọc, không cần thiết lập thêm.
- Nhược điểm: thông tin nhạy cảm lộ trong git history, khi cần thay đổi phải sửa nhiều file, không phân biệt được service nào được phép truy cập service nào.

b. Chuyển sang Kubernetes Secrets

Nhóm tách thông tin xác thực ra thành các Secret riêng biệt theo từng service (airflow-secret, clickhouse-secret, minio-secret, postgres-secret), các pod truy cập qua secretRef hoặc secretKeyRef thay vì đọc trực tiếp từ ConfigMap.

Đánh đổi:

- Ưu điểm: tách biệt cấu hình và thông tin xác thực, pod chỉ nhìn thấy Secret của chính mình, dễ thay đổi mà không cần rebuild image.
- Nhược điểm: Kubernetes Secret mặc định chỉ mã hóa base64 (không phải mã hóa thật sự), cần bổ sung encryption at rest ở cấp etcd nếu muốn bảo mật thực sự.

5.7.3. Giải pháp cuối cùng

a. Phân quyền tối thiểu qua RBAC

Thay vì cấp quyền admin cho Airflow Scheduler, nhóm tạo Role deployment-scaler với đúng ba quyền cần thiết: get, patch, update trên deployments và deployments/scale trong namespace marketing. ServiceAccount của Airflow Scheduler được bind vào Role này và không có quyền nào khác — không đọc được Secret, không tạo/xóa Pod, không can thiệp vào namespace khác.

b. Nhật ký vận hành tập trung quan Airflow Remote Logging

Toàn bộ log của Airflow task được đẩy về MinIO bucket s3://marketing-datalake/airflow-logs/ tổ chức theo cấu trúc dag_id/run_id/task_id/attempt.log. Nhờ đó, lịch sử thực thi của mọi DAG run được lưu bền vững và có thể tra cứu lại bất kỳ lúc nào, ngay cả khi pod Airflow đã bị xóa và tạo lại.

c. Dead Letter Queue làm audit trail cho luồng thời gian thực

Kafka Connect được cấu hình errors.tolerance: all kết hợp errors.deadletterqueue.topic.name: dlq_speed_layer. Mọi message không thể ghi vào ClickHouse — do lỗi schema hay mất kết nối — đều được chuyển vào topic DLQ thay vì bị bỏ

qua im lặng. Đây là cơ chế audit cơ bản cho luồng thời gian thực, giúp phát hiện dữ liệu bị rớt mà không làm gián đoạn toàn bộ pipeline.

d. Quản lý vòng đời dữ liệu qua TTL

Bảng `rt_*` trong ClickHouse được gán TTL một ngày: dữ liệu tạm thời từ Speed Layer tự động bị xóa sau khi Batch Layer ghi kết quả chính xác vào bảng tương ứng. Cơ chế này đảm bảo dữ liệu không tích lũy vô thời hạn và giữ cho Data Warehouse luôn phản ánh đúng nguồn tin cậy duy nhất.

5.7.4. Điểm rút ra

a. Hiểu biết kỹ thuật

Bảo mật và quản trị trong hệ thống phân tán không chỉ là vấn đề mật khẩu mạnh hay TLS — mà còn là bài toán ai được phép làm gì với tài nguyên nào, dữ liệu đi đâu khi có lỗi, và làm thế nào để truy vết được toàn bộ hành trình dữ liệu khi xảy ra sự cố. Môi trường phát triển cục bộ cho phép bỏ qua nhiều lớp bảo mật, nhưng việc hiểu rõ những gì đang bị bỏ qua là điều kiện tiên quyết để không mang những thiếu sót đó lên production.

b. Best practice

Một số nguyên tắc nhóm đúc kết được:

- Tách thông tin xác thực ra khỏi code và cấu hình ngay từ đầu, dù chỉ là môi trường phát triển.
- Cấp quyền tối thiểu cho từng service — một DAG lỗi không nên có khả năng phá hủy tài nguyên ngoài phạm vi của nó.
- Luôn có cơ chế lưu vết cho dữ liệu bị lỗi (DLQ, log tập trung) thay vì để lỗi biến mất im lặng.

c. Đề xuất

Ba cải tiến ưu tiên khi chuyển lên môi trường production: thay thế Kubernetes Secrets thuần bằng giải pháp quản lý bí mật chuyên dụng như HashiCorp Vault hoặc External Secrets Operator, bật TLS cho toàn bộ kết nối service-to-service (Kafka SASL_SSL, ClickHouse HTTPS, MinIO TLS), và bổ sung NetworkPolicy để giới hạn luồng traffic nội bộ giữa các pod theo nguyên tắc chỉ cho phép kết nối khi thực sự cần thiết.

CHƯƠNG 6. NHẬN XÉT, ĐÁNH GIÁ VÀ HƯỚNG PHÁT TRIỂN

6.1. Nhận xét, đánh giá

Hệ thống cho thấy những lợi ích mà một hệ thống Big Data đem lại như khả năng lưu trữ, tìm kiếm, biểu diễn lượng lớn dữ liệu, khả năng mở rộng khi lượng tài nguyên hiện tại không đủ, khả năng chịu lỗi trong một mạng phân tán khi có những thành phần trong mạng gặp trục trặc. Đây là những khả năng mà các hệ thống truyền thống không có hoặc khả năng đáp ứng còn hạn chế.

Bên cạnh đó, hệ thống của nhóm có một số nhược điểm. Việc sử dụng spark của nhóm không khai thác được tối đa hệ thống. Lượng dữ liệu được thu thập khá ít, hoàn toàn có thể chạy trong 1 máy. Ngoài ra luồng thực hiện của hệ thống vẫn khá rời rạc, một số bước tải dữ liệu vẫn thực hiện bằng cách gõ code thủ công mà chưa được tự động hóa.

6.2. Hướng phát triển

Do quá trình crawl dữ liệu được thực hiện trên một luồng nên tốc độ có thể được tăng tốc bằng lập trình đa luồng.

Sử dụng Spark Streaming để phân tích và cải thiện tốc độ ghi dữ liệu.

Thêm quy trình CI/CD để giảm thiểu lỗi trong quá trình kiểm thử và deployment thủ công.

DANH MỤC TÀI LIỆU THAM KHẢO

1. <https://demanejar.github.io/posts/mode-in-spark/>
2. Bài giảng “Lưu trữ và xử lý dữ liệu lớn” – TS. Trần Việt Trung
3. <https://spark.apache.org/docs/latest/running-on-kubernetes.html>
4. <https://clickhouse.com/docs/install/docker>
5. <https://superset.apache.org/admin-docs/installation/installation-methods>
6. <https://viblo.asia/p/trien-khai-kafka-kraft-mode-va-kafkaui-bang-docker-compose-co-the-ap-dung-cho-moi-truong-production-gjLN0eyd432>
7. Giáo trình “Tổng quan về dữ liệu lớn (Big Data)” – Ks. Nguyễn Công Hoan – Trung Tâm Thông Tin Khoa học thống kê (Viện KHTK)